

We can define the sum of the bits in the specified locations with one line. We store it in a variable called `xor_result`, because taking a sum mod 2 is also called the *exclusive OR* or *XOR operation*. If you have studied formal logic, you may have encountered XOR before—it has a logical definition and an equivalent mathematical definition; here we will use the mathematical definition. Since we are working with a short bit-string, we don't sum the 4th, 6th, 8th, and 10th bits (since those don't exist), but instead sum the 2nd and 3rd bits:

```
xor_result = (bits[1] + bits[2]) % 2
```

Then, we can take out the rightmost element of the bits easily with Python's handy `pop()` function, storing the result in a variable called `output`:

```
output = bits.pop()
```

We can then insert our sum with the `insert()` function, specifying position 0 since we want it to be on the left side of our list:

```
bits.insert(0,xor_result)
```

Now let's put it all together into one function that will return two outputs: a pseudorandom bit and a new state for the `bits` series ([Listing 5-9](#)).

```
def feedback_shift(bits):  
    xor_result = (bits[1] + bits[2]) % 2  
    output = bits.pop()  
    bits.insert(0,xor_result)  
    return(bits,output)
```

[Listing 5-9](#): A function that implements an LFSR, completing our goal for this section

Just as we did with the LCG, we can create a function that will generate an entire list of our output bits:

```
def feedback_shift_list(bits_this):  
    bits_output = [bits_this.copy()]  
    random_output = []  
    bits_next = bits_this.copy()  
    while(len(bits_output) < 2**len(bits_this)):  
        bits_next,next = feedback_shift(bits_next)
```
